



```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
In [2]: dataset=pd.read_csv("Social_Network_Ads.csv")
```

```
In [3]: dataset
```

```
Out[3]:
```

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
...	...	...	...	...	...
395	15691863	Female	46	41000	1
396	15706071	Male	51	23000	1
397	15654296	Female	50	20000	1
398	15755018	Male	36	33000	0
399	15594041	Female	49	36000	1

400 rows × 5 columns

```
In [4]: dataset = dataset.drop(columns=['User ID'])
```

```
In [5]: dataset=pd.get_dummies(dataset,drop_first=True)
```

```
In [6]: dataset
```

```
Out[6]:
```

	Age	EstimatedSalary	Purchased	Gender_Male
0	19	19000	0	True
1	35	20000	0	True
2	26	43000	0	False
3	27	57000	0	False
4	19	76000	0	True
...	...	...	...	...
395	46	41000	1	False
396	51	23000	1	True
397	50	20000	1	False
398	36	33000	0	True
399	49	36000	1	False

400 rows × 4 columns

```
In [7]: x=dataset[['Age', 'EstimatedSalary', 'Gender_Male']]
        y=dataset[['Purchased']]
```

```
In [8]: from sklearn.model_selection import train_test_split
        x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_s
```

```
In [10... from sklearn.linear_model import LogisticRegression
        clf = LogisticRegression(random_state=0)
        clf.fit(x, y)
```

C:\Users\Gayathri R\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\utils\validation.py:1300: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().

y = column_or_1d(y, warn=True)

C:\Users\Gayathri R\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\linear_model_logistic.py:469: ConvergenceWarning: lbfgs failed to converge (status=1):

STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

n_iter_i = _check_optimize_result(

Out[10]:

LogisticRegression

LogisticRegression(random_state=0)

```
In [15... from sklearn.model_selection import GridSearchCV
param_grid = {'C': [0.001, 0.01, 0.1, 1, 10, 100],
              'penalty': ['l1', 'l2'],
              'solver': ['liblinear', 'saga']}
grid=GridSearchCV(LogisticRegression(),
                  param_grid=param_grid,
                  refit=True,
                  verbose=3,
                  n_jobs=-1)
grid.fit(x_train, y_train)
```

Fitting 5 folds for each of 24 candidates, totalling 120 fits

C:\Users\Gayathri R\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\utils\validation.py:1300: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().
y = column_or_1d(y, warn=True)

Out[15]:

GridSearchCV

estimator: LogisticRegression

LogisticRegression

```
In [16... regressor=grid.cv_results_
grid_predictions=grid.predict(x_test)
from sklearn.metrics import confusion_matrix
cm=confusion_matrix(y_test,grid_predictions)
from sklearn.metrics import classification_report
clf_report=classification_report(y_test,grid_predictions)
```

```
In [17... from sklearn.metrics import f1_score
f1_macro=f1_score(y_test,grid_predictions,average='macro')
print("The f1-score value for best parameter{}:",format(grid.best_params_
```

The f1-score value for best parameter{}: {'C': 100, 'penalty': 'l1', 'solver': 'liblinear'} 0.7619047619047619

```
In [18... print("The confusion matrix:\n",cm)
```

The confusion matrix:
[[48 5]
[11 16]]

```
In [19... print("The report:\n",clf_report)
```

The report:

	precision	recall	f1-score	support
0	0.81	0.91	0.86	53
1	0.76	0.59	0.67	27
accuracy			0.80	80
macro avg	0.79	0.75	0.76	80
weighted avg	0.80	0.80	0.79	80

```
In [23... from sklearn.metrics import roc_auc_score
roc_auc_score(y, clf.predict_proba(x)[: , 1])
```

Out[23]: 0.9140567603602623

```
In [27... from sklearn.metrics import accuracy_score
y_pred = [0, 2, 1, 3]
y_true = [0, 1, 2, 3]
accuracy_score(y_true, y_pred)
accuracy_score(y_true, y_pred, normalize=False)
```

Out[27]: 2.0

```
In [25... from sklearn.metrics import balanced_accuracy_score
y_true = [0, 1, 0, 0, 1, 0]
y_pred = [0, 1, 0, 0, 0, 1]
balanced_accuracy_score(y_true, y_pred)
```

Out[25]: 0.625

In []: